1) ***Utilizando como función hash f(x)= x mod 1301, conceptualice un archivo directo de 1300 posiciones y ubique los siguientes valores: 10168 - 21362 – 8515 – 7993 -32434 – 29786 – 35175 – 50255 – 32305 – 24971 – 10177 – 53101 – 12901. Indiquen la cantidad máxima de claves, y cómo calcularía el factor de ocupación. ¿Existieron colisiones? ¿Cómo las resolvió?***

Posición física	Clave Primaria
1061	10168
546	21362
709	8515
187	7993
121	32434
1164	29786
48	35175
817	50255
1081	32305
252	24971
170	10177
1061	53101
1192	12901

La función hash tiene como máxima cantidad de claves 1301, desde 0 hasta 1300

Factor de ocupación: 13/1300 = 0.01

El registro con clave 53101 colisiona con el registro con clave 10168, para resolver esta colisión aplicamos el criterio de direccionamiento cerrado en el que se realiza un sondeo lineal y ubicando el primero en la posición 1061 + 1 = 1062

2) ***Para los valores del ejercicio 1, proponga otras dos funciones hash, calcule las claves e indique si existieron colisiones y cómo las resolvería. Considere un archivo de 200 posiciones.***

Primero usamos el método de multiplicación para hashing

m = 200 A = 0.6180339

$$\mathrm{h(k)} = \lfloor m * \{k * A\} \rfloor = 33$$

$h = 10168 \rightarrow h(k) = 33$

$k = 21362 \rightarrow \mathrm{h(k)} = 88$

$k = 8515 \rightarrow h(k) = 111$

$k = 7993 \rightarrow h(k) = 188$

$k = 32434 \rightarrow h(k) = 62$

$k = 29786 \rightarrow h(k) = 151$

$k = 35175 \rightarrow h(k) = 68$

$k = 50255 \rightarrow h(k) = 58$

$k = 32305 \rightarrow h(k) = 117$

$k = 24971 \rightarrow h(k) = 184$

$k = 10177 \rightarrow h(k) = 146$

$k = 53101 \rightarrow h(k) = 43$

$k = 12901 \rightarrow h(k) = 51$

No hubieron colisiones

Como segundo metodo de hashing se utilizo el plegado por desplazamiento con m = 200 y

$$h_{shift}(k) = \left(\sum B_i\right) mod\ m$$

$k = 10168 \rightarrow h(k) = 70$

$k = 21362 \rightarrow h(k) = 77$

$k = 8515 \rightarrow h(k) = 100$

$k = 7993 \rightarrow h(k) = 172$

$k = 32434 \rightarrow h(k) = 61$

$k = 29786 \rightarrow h(k) = 185$

$k = 35175 \rightarrow h(k) = 129$

$k = 50255 \rightarrow h(k) = 62$

$k = 32305 \rightarrow h(k) = 31$

$k = 24971 \rightarrow h(k) = 122$

$k = 10177 \rightarrow h(k) = 79$

$k = 53101 \rightarrow h(k) = 37$

$k = 12901 \rightarrow h(k) = 31$

Se produjo una colisión en la clave 12901 y la clave 32305 la cual solucionamos aplicando el criterio de direccionamiento cerrado, realizando un sondeo lineal y posicionando la clave 12901 en la posición 32

3) ***Suponga un archivo de organización directa de 512 posiciones. Usando una función hash con método de compresión y seleccionando el número primo adecuado, calcule las posiciones donde irían los siguientes registros alfanuméricos: limón – gato – caradura – vodka – ojo – ojota – caradura - oh***

Palabra	XOR Result	Índice (mod 521)
limon	105	105
gato	29	29
caradura	19	19
vodka	119	119
ojo	106	106
ojota	127	127

caradura	19	19
oh	7	7

Al igual que en los ejercicios previos para solucionar la colisión en la segunda aparición de la palabra “caradura” se la posiciona en la posición física 20.

Informe ejercicio de programación N°1

Para la persistencia del sistema usamos un archivo binario (Un .dat con registros de longitud fija). Y además creamos la tabla hash con 12007 posiciones, la cual se carga con el DNI de cada persona, y la posición física, se crea en memoria para evitar accesos innecesarios al disco.
Cada registro del archivo, visto desde el lenguaje C, tiene esta estructura:

```
typedef struct {
    int dni;
    char nombre[MAX];
    char sexo;
    unsigned short int edad;
    char categoria[MAX];
    char tiempo[9];
    unsigned short int activo;
} Treg;
```

Para la manipulación del archivo se usa uno de acceso directo que se inicializa con 12000 registros con los campos "vacíos" junto con la tabla de hash, con 12007 posiciones, dado que es el primer primo mayor al tamaño físico del archivo.

```
#define MAX_CORREDORES 12000
#define TAM_HASH 12007
```

Como método de hashing utilizamos Módulo. Es decir, para obtener la posición a guardar el registro, siendo esta el número de corredor, al DNI de la persona se le aplica módulo del siguiente número primo al tamaño del archivo, pero para que sea más eficiente y genere menos colisiones.

```
int hash_function(int dni) {

    return dni % TAM_HASH;
}
```

Las altas se basan en crear un registro de corredor y llenarlo con los datos necesarios, siendo estos: DNI, sexo, nombre, edad y categoría.
Luego se le aplica el método de hashing para cargarlo en la tabla de hash y posteriormente en el archivo. En caso de que el hash esté ocupado, se realiza como solución a la colisión, el método de sondeo lineal, es decir, se busca el siguiente espacio disponible en la tabla de hash. Esta solución puede ser ineficiente, dado que recorre posición por posición y en caso de que no haya lugar disponible, la recorre por completo.

```
int resolver_colision_sondeo_lineal(int pos_inicial, int dni, int*
encontrado) {
    int i;

    // Primer tramo: desde la posicion hash hasta el final de la tabla.
    for (i = pos_inicial; i < TAM_HASH; i++) {
        if (tabla_hash[i] == dni) {
            *encontrado = 1;
            return i;
        }

        if (tabla_hash[i] == 0) {
            *encontrado = 0;
            return i;
```

```
        }
    }

    // Segundo tramo: desde el inicio hasta la posicion hash - 1.
    for (i = 0; i < pos_inicial; i++) {
        if (tabla_hash[i] == dni) {
            *encontrado = 1;
            return i;
        }

        if (tabla_hash[i] == 0) {
            *encontrado = 0;
            return i;
        }
    }

    *encontrado = 0;
    return -1;
}
/ Busca o inserta un DNI en la tabla hash.
// crear=0: solo busca. crear=1: inserta cuando no existe.
// Retorna el indice hash (que coincide con la posicion fisica).
int resolver_posicion_hash(int dni, int crear) {
    int pos_inicial = hash_function(dni);
    int encontrado;
    int idx = resolver_colision_sondeo_lineal(pos_inicial, dni,
&encontrado);

    if (idx == -1) {
        return -1;
    }

    if (encontrado) {
        return idx;
    }

    if (!crear) {
        return -1;
    }

    tabla_hash[idx] = dni;
    return idx;
}
int alta_corredor(FILE* f, Treg corredor) {
    int pos_fisica = resolver_posicion_hash(corredor.dni, 0);
    Treg existente;
```

```
    if (pos_fisica != -1) {
        fseek(f, pos_fisica * sizeof(Treg), SEEK_SET);
        if (fread(&existente, sizeof(Treg), 1, f) != 1) {
            return -1;
        }

        if (existente.activo == ACTIVO) {
            printf("Error: El DNI ya esta registrado\n");
            return -1;
        }

        fseek(f, pos_fisica * sizeof(Treg), SEEK_SET);
        fwrite(&corredor, sizeof(Treg), 1, f);
        fflush(f);
        printf("Corredor registrado en posicion: %d\n", pos_fisica);
        return 0;
    }

    pos_fisica = resolver_posicion_hash(corredor.dni, 1);
    if (pos_fisica == -1) {
        printf("Error: Tabla hash sin espacio\n");
        return -1;
    }

    fseek(f, pos_fisica * sizeof(Treg), SEEK_SET);
    fwrite(&corredor, sizeof(Treg), 1, f);
    fflush(f);
    printf("Corredor registrado en posicion: %d\n", pos_fisica);
    return 0;
}
```

Las modificaciones serían el cargado del tiempo realizado por el corredor en la maratón, en la que se ingresa su DNI, se obtiene la posición del registro (usando el método de hash explicado) y se modifica el campo tiempo al ingresarlo en formato de horas, minutos y segundos (hh:mm:ss). Luego tambien se puede modificar su nombre, sexo, edad y categoria cargada al darle de alta.

```
// Modifica un corredor existente manteniendo su tiempo previo si ya lo
tenia.
int modificar_corredor(FILE* f, int dni, Treg nuevo_datos) {
    int posicion = resolver_posicion_hash(dni, 0);

    if (posicion != -1) {
        // Mantener el tiempo anterior si existe
        Treg anterior;
        fseek(f, posicion * sizeof(Treg), SEEK_SET);
        if (fread(&anterior, sizeof(Treg), 1, f) != 1) {
            return -1;
        }
```

```
        if (anterior.dni != dni || anterior.activo != ACTIVO) {
            return -1;
        }

        if (strlen(anterior.tiempo) > 0) {
            strcpy(nuevo_datos.tiempo, anterior.tiempo);
        }

        fseek(f, posicion * sizeof(Treg), SEEK_SET);
        fwrite(&nuevo_datos, sizeof(Treg), 1, f);
        fflush(f);
        return 0;
    }
    return -1;
}
```

En caso de las bajas, se ingresa un DNI, se obtiene su posición, se elimina tanto de la tabla de hash (se pone en 0 el DNI) y marca el campo activo en falso del registro del archivo.

```
// Baja logica: marca inactivo en archivo y elimina DNI del indice hash.
int baja_corredor(FILE* f, int dni) {
    int posicion = resolver_posicion_hash(dni, 0);
    Treg corredor;

    if (posicion != -1) {
        fseek(f, posicion * sizeof(Treg), SEEK_SET);
        if (fread(&corredor, sizeof(Treg), 1, f) != 1) {
            return -1;
        }

        if (corredor.dni != dni || corredor.activo != ACTIVO) {
            return -1;
        }

        corredor.activo = INACTIVO;
        fseek(f, posicion * sizeof(Treg), SEEK_SET);
        fwrite(&corredor, sizeof(Treg), 1, f);
        fflush(f);

        eliminar_dni_de_hash(dni);
        return 0;
    }
    return -1;
}
```

Por último, los listados que muestra el sistema son de tiempo general y por categoría, en ambos casos se revisa el campo activo del registro, para verificar si es un corredor válido.

El primero recorre la tabla hash secuencialmente para luego posicionarse en el correspondiente registro del archivo, leerlo y mostrar el número de corredor y su tiempo registrado durante la maratón, ordenado por el propio número.
El segundo listado, imita la implementación del primero y muestra los corredores que tengan la categoría solicitada, ya sea, media maratón (21 km) o final (42 km), imprimiendo su número y tiempo registrado.
Esta solución no es eficiente para realizar el listado de tiempos generales, ni para el listado por categoría.
Esto se debe a que en general, en el peor de los casos el archivo está lleno, por lo tanto, se recorre secuencialmente, desaprovechando que el propio archivo sea de acceso directo.
Para el listado por categoria, tambien se debe recorrer secuencialmente el archivo, ya que debe revisar el campo categoria de cada registro, para analizar si lo debe mostrar por pantalla o no, por lo tanto desaprovecha el acceso directo.

Si se tomara el tiempo con sensores RFID una de las cosas que se verían modificadas es el acceso al archivo y las operaciones realizadas sobre este. Teniendo en cuenta el corto intervalo de tiempo entre solicitudes las operaciones sobre el archivo deben realizarse en batch, por lo que primero se realizan en memoria para posteriormente (cuando termine la carrera o en un momento que se determine) persistirlas en el archivo.

Informe ejercicio de programación N°2

Para la creación del archivo principal, se implementó una función que crea el archivo con M registros de empleados vacíos que tienen como campos los datos personales, nombre y apellido, edad, dirección, teléfono, y fecha de ingreso y un campo que determina el estado de los datos, siendo estos, 0, 1 o 2, determinando, el primero que el registro está disponible para almacenar, el segundo que el registro está ocupado con datos válidos y el tercero que tiene datos inválidos/eliminados.

```
typedef struct {
    char nyap[80];  // Clave de acceso: Apellido y Nombre
    char direccion[100];
    char telefono[20];
    char fecha_ingreso[11];  // Formato DD/MM/AAAA
    int estado;              // 0: Libre, 1: Ocupado, 2: Borrado
} TRegEmpleado;
```

Para la tabla hash, se utilizó el método de hashing Modulo aplicado al campo de nombre y apellido, con la variante para cadenas, es decir, sumando carácter a carácter y posteriormente aplicarle la OPERACION modulo a la sumatoria resultante con el tamaño de la tabla hash.

```
int funcion_hash(char* clave) {
    int suma = 0;
    for (int i = 0; clave[i] != '\0'; i++) {
        suma += clave[i];
    }
    return suma % M;  // Retorna un índice entre 0 y M-1
}
```

Para la solución de colisiones, se utiliza una zona de desbordamiento. Para esta solución dicha zona de desbordamiento se implementó como un archivo separado que vamos a denominar *archivo de overflow* en donde se almacenaran las colisiones. Este archivo tiene registros de longitud fija análogos a los del archivo principal. Los pasos que se desarrollaron para resolver las colisiones son los siguientes:
Primero se analiza si el hash obtenido por la función aplicada está ocupado verificando el estado del registro. Si no está ocupado significa que se puede realizar una inserción en esa posición, de lo contrario utilizamos el archivo de overflow para ir a la misma posición determinada por la función hash en donde repetimos este proceso. Comprobamos si la posición calculada está ocupada también en el archivo de overflow, si es así, realizamos un sondeo lineal al archivo de overflow para encontrar una posición para insertar.

Para el alta de un empleado, se le aplica el método de hash explicado, obteniendo el índice de la tabla de hash, se revisa si la posición está ocupada para analizar si se trata de una colisión o no. En caso de que sea una posición libre, se inserta el registro en el archivo principal en el índice indicado por el hash y se marca en el campo Estado del registro en 1, que anteriormente estaba en 0 (Libre/disponible) o en 2 (eliminado).
Si la posición está ocupada, se revisa la misma, pero en la zona de desbordamiento y se realiza el método de solución de colisión ya explicado.

```
int resolver_colision(FILE* archOverflow, int pos_inicial, char* clave,
int* encontrado) {
    TRegEmpleado actual;
```

```
    if (strcmp(tabla_hash[pos_inicial].nyap, clave) == 0) {
        *encontrado = 1;
        return pos_inicial;
    }
    // Primer tramo: desde la posicion hash hasta el final de la tabla.
    for (int i = pos_inicial; i < TAM_HASH; i++) {
        fseek(archOverflow, pos_inicial * sizeof(TRegEmpleado),
SEEK_SET);
        fread(&actual, sizeof(TRegEmpleado), 1, archOverflow);
        if (strcmp(actual.nyap, clave) == 0) {
            *encontrado = 1;
            return i;
        }
        if (actual.estado == 0 || actual.estado == 2) {
            *encontrado = 0;
            return i;
        }
    }

    // Segundo tramo: desde el inicio hasta la posicion hash - 1.
    for (i = 0; i < pos_inicial; i++) {
        fseek(archOverflow, pos_inicial * sizeof(TRegEmpleado),
SEEK_SET);
        fread(&actual, sizeof(TRegEmpleado), 1, archOverflow);
        if (strcmp(actual.nyap, clave) == 0) {
            *encontrado = 1;
            return i;
        }
        if (actual.estado == 0 || actual.estado == 2) {
            *encontrado = 0;
            return i;
        }
    }

    *encontrado = 0;
    return -1;
}

void alta_empleado(TRegEmpleado nuevo) {
    FILE* fp = fopen("principal.dat", "r+b");
    FILE* fo = fopen("overflow.dat", "r+b");

    int pos_home = funcion_hash(nuevo.nyap);
    TRegEmpleado actual;
    if (strcmp(tabla_hash[pos_home].nyap, nuevo.nyap) != 0 &&
tabla_hash[pos_home].nyap[0] != '\0') {
```

```
        int encontrado;
        int idx = resolver_colision(fo, pos_home, nuevo.nyap,
&encontrado);
        // escribo en archivo overflow
        fseek(fo, (pos_home + idx) * sizeof(TRegEmpleado), SEEK_SET);
        fwrite(&nuevo, sizeof(TRegEmpleado), 1, fo);
    } else {
        // escribo en archivo principal y tabla hash
        strcpy(tabla_hash[pos_home].nyap, nuevo.nyap);
        fseek(fp, pos_home * sizeof(TRegEmpleado), SEEK_SET);
        fwrite(&nuevo, sizeof(TRegEmpleado), 1, fp);
    }

    fclose(fp);
    fclose(fo);
}
```

Para la baja o eliminación de registros, ingresando el nombre y apellido, se obtiene el hash correspondiente y con este se revisa si el registro buscado está en el archivo principal al analizar la tabla de hash. En caso de estarlo, se accede al archivo y se marca el campo estado en 2, es decir datos eliminados/borrados. En el caso de que no coincidan las claves, se lo busca secuencialmente a partir de la misma posición en el archivo de overflow y se resuelve de la misma manera, marcando el campo estado en 2.

```
void baja_empleado(char* clave) {
    FILE* fp = fopen("principal.dat", "r+b");
    FILE* fo = fopen("overflow.dat", "r+b");

    int pos_home = funcion_hash(clave);
    TRegEmpleado actual;
    // me fijo si esta en la tabla hash antes de acceder al archivo
principal
    // si no esta, puede ser que este en el archivo de overflow
    if (strcmp(tabla_hash[pos_home].nyap, clave) == 0) {
        fseek(fp, pos_home * sizeof(TRegEmpleado), SEEK_SET);
        fread(&actual, sizeof(TRegEmpleado), 1, fp);
        if (actual.estado == 1 && strcmp(actual.nyap, clave) == 0) {
            actual.estado = 2;
            fseek(fp, pos_home * sizeof(TRegEmpleado), SEEK_SET);
            fwrite(&actual, sizeof(TRegEmpleado), 1, fp);
            printf("Baja lógica en principal.\n");
        } else {
            if (actual.estado == 2 && strcmp(tabla_hash[pos_home].nyap,
clave) == 0) {
                printf("Posicion invalida \n");
            } else {
                int encontrado;
                int idx = resolver_colision(fo, pos_home, clave,
&encontrado);
                if (encontrado) {
```

```
                    fseek(fo, (pos_home + idx) * sizeof(TRegEmpleado),
SEEK_SET);
                    fwrite(&actual, sizeof(TRegEmpleado), 1, fo);
                    printf("Baja lógica en overflow\n");
                }
            }
        }
    } else {
        fseek(fo, pos_home * sizeof(TRegEmpleado), SEEK_SET);
        fread(&actual, sizeof(TRegEmpleado), 1, fo);
        int encontrado;
        int idx = resolver_colision(fo, pos_home, clave, &encontrado);
        if (encontrado) {
            fseek(fo, (pos_home + idx) * sizeof(TRegEmpleado), SEEK_SET);
            fwrite(&actual, sizeof(TRegEmpleado), 1, fo);
            printf("Baja lógica en overflow\n");
        }
    }
    fclose(fp);
    fclose(fo);
}
```

Por último, para las modificaciones, se ingresa el nombre y apellido, se obtiene el hash correspondiente, se accede al archivo principal y se revisa el campo estado del registro. En caso de que sea 1, se permite la modificación de los datos. En caso de que sea 0 o 2 y coincide la clave, se muestra que la posición a modificar es invalida. Por último, en el caso de que tenga estado 2 y no coincida la clave, se procede a buscar el registro en el área de desbordamiento, accediendo directamente a la dirección obtenida del hash o buscando secuencialmente a partir de esta para encontrar el registro buscado para la modificación, que al encontrarlo también se revisa el campo estado para analizar si es una modificación valida.

```
void modificar_empleado(char* clave, char* nueva_dir, char* nuevo_tel) {
    FILE* fp = fopen("principal.dat", "r+b");
    FILE* fo = fopen("overflow.dat", "r+b");

    int pos_home = funcion_hash(clave);
    TRegEmpleado actual;
    fseek(fp, pos_home * sizeof(TRegEmpleado), SEEK_SET);
    fread(&actual, sizeof(TRegEmpleado), 1, fp);

    if (strcmp(tabla_hash[pos_home].nyap, nuevo.nyap) == 0 &&
actual.estado == 1) {
        // lo encontre en el archivo principal, debo modificar.
        strcpy(actual.direccion, nueva_dir);
        strcpy(actual.telefono, nuevo_tel);
        fseek(fp, pos_home * sizeof(TRegEmpleado), SEEK_SET);
        fwrite(&actual, sizeof(TRegEmpleado), 1, fp);
        printf("Modificado en principal.\n");

    } else {
```

```
        if (actual.estado == 0 || (actual.estado == 2 &&
strcmp(tabla_hash[pos_home].nyap, nuevo.nyap) == 0))
            printf("Posicion Invalidad.\n");
        else {
            // debo buscar en overflow
            int encontrado;
            int idx = resolver_colision(fo, pos_home, nuevo.nyap,
&encontrado);
            if (encontrado) {
                strcpy(actual.direccion, nueva_dir);
                strcpy(actual.telefono, nuevo_tel);
                fseek(fo, (pos_home + idx) * sizeof(TRegEmpleado),
SEEK_SET);
                fwrite(&actual, sizeof(TRegEmpleado), 1, fo);
                printf("Modificado en overflow\n");
            }
        }
    }

    fclose(fp);
    fclose(fo);
}
```